GUÍA DE INSTALACIÓN · COMPLEMENTO DEL CURSO

Instalar REA + Ghidra en tu PC con Claude Code y Antigravity

Guía paso a paso, en español, para dejar listo en tu ordenador todo lo que necesita el curso *"Cómo hacer ingeniería inversa de cualquier programa con Claude Opus 5.5"*: Claude Code, REA y Ghidra, trabajando dentro de Antigravity.

Repositorio	`github.com/edgar-manuel/prueba-rea`
Herramientas	Antigravity · Claude Code · REA (`rea-agents`) · Ghidra 12.1.x · JDK 21
Sistemas	Windows 10/11, macOS y Linux
Tiempo estimado	30–45 minutos (la mayor parte, descargas)

> **Cómo leer esta guía**
> Cada paso tiene lo que **vas a hacer**, los **comandos** listos para copiar (con variantes para Windows y para macOS/Linux) y una **comprobación** para saber que ha salido bien antes de pasar al siguiente. Al final hay una tabla de **problemas típicos**.

Qué hay ya hecho en la nube

En la sesión de Claude Code en la nube (desde el móvil) quedó configurado:

Pieza	Estado
REA 6.7.0 (CLI `rea` + servidor MCP)	Instalado y conectado (`rea doctor` → `healthy: true`)
Skill `reverse-engineer-anything`	Instalada en Claude Code
`CLAUDE.md` con las "reglas de la casa"	En el repositorio
`.claude/hooks/session-start.sh` + `.claude/settings.json`	Reinstalan REA en cada sesión en la nube
Ghidra	**No instalado**: la red de la nube bloquea la descarga desde GitHub

Por eso Ghidra se instala en tu PC. El hook de arranque **solo actúa en la nube** (comprueba `CLAUDE_CODE_REMOTE`), así que en tu PC no hace nada y no interfiere: allí lo instalas tú una vez siguiendo esta guía.

El mapa de la instalación

Paso	Qué haces	Comprobación
01 · Requisitos	Node.js 22+, Git y JDK 21	`node -v`, `git --version`, `java -version`
02 · Antigravity + Claude Code	Instalar Claude Code e iniciar sesión	`claude --version`
03 · El repositorio	Clonar `prueba-rea` y abrirlo en Antigravity	Ves `CLAUDE.md` en el explorador
04 · REA	Instalar la CLI y registrarla en Claude Code	`rea --help`
05 · Ghidra	Descargar, descomprimir y apuntar `GHIDRA_INSTALL_DIR`	`rea doctor` → ghidra OK
06 · Verificar	Comprobar las herramientas MCP desde Claude	`/mcp` → `rea: connected`
07 · Primera prueba	Analizar un programa **tuyo**	Pseudocódigo de `main`

PASO 01

Los requisitos

Node.js, Git y Java 21

REA es un paquete de Node.js, así que necesitas **Node.js 22 o superior** con npm. La última versión de REA (6.8.0) pide **Node 22.23.4 o más nuevo**; instala la versión LTS actual y te ahorras problemas. Ghidra está escrito en Java y necesita un **JDK 21 de 64 bits** (no basta con un JRE ni sirve Java 17). Git te hace falta para clonar el repositorio.

WINDOWS (PowerShell)

```
# Con winget (viene con Windows 10/11)
winget install OpenJS.NodeJS.LTS
winget install Git.Git
winget install EclipseAdoptium.Temurin.21.JDK

# Cierra y vuelve a abrir la terminal para que se carguen los cambios
```

MACOS (Terminal)

```
# Con Homebrew (https://brew.sh)
brew install node git
brew install --cask temurin@21
```

LINUX (Ubuntu/Debian)

```
# Node.js LTS desde NodeSource
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs git openjdk-21-jdk unzip
```

COMPROBACIÓN

```
node -v          # v22.23.4 o superior (o v24.x)
npm -v
git --version
java -version    # debe decir "21"
```

> **En pocas palabras**
> Node para REA, Java 21 para Ghidra, Git para el repositorio. Si `java -version` dice otra versión, Ghidra no arrancará.

PASO 02

Antigravity y Claude Code

El editor y el agente

Antigravity es el editor de Google basado en VS Code. Claude Code funciona dentro de él de dos formas: como **extensión** (panel lateral) o como **CLI** en la terminal integrada. La CLI es la que usa la guía del curso y la que necesita REA, así que instálala siempre; la extensión es opcional.

1. Descarga e instala Antigravity desde su web oficial (`antigravity.google`) si aún no lo tienes.

2. Abre la terminal integrada: menú **Terminal → New Terminal** (o `Ctrl+`` `).

3. Instala Claude Code:

TERMINAL

```
# Instalador nativo — macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash

# Instalador nativo — Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex

# Alternativa con npm (cualquier sistema)
npm install -g @anthropic-ai/claude-code
```

4. Inicia sesión con tu cuenta de Claude:

```
claude          # la primera vez abre el navegador para iniciar sesión
```

5. Opcional — extensión: en Antigravity, abre Extensiones (`Ctrl+Shift+X`), busca **“Claude Code”** (publicada por Anthropic) e instálala. Si no aparece en el catálogo, no pasa nada: la CLI en la terminal integrada hace exactamente lo mismo.

COMPROBACIÓN

```
claude --version
```

> **En pocas palabras**
> Antigravity es el taller; Claude Code, el cerebro. Usa la CLI en la terminal integrada de Antigravity.

PASO 03

El repositorio

Trae las reglas y la configuración

El repositorio `prueba-rea` ya trae el `CLAUDE.md` con las reglas de ingeniería inversa. Clónalo y ábrelo como carpeta de trabajo para que Claude Code las cargue.

TERMINAL

```
git clone https://github.com/edgar-manuel/prueba-rea.git
cd prueba-rea
git checkout ccr-e331b2df-ws5vze   # la rama donde está todo lo de la nube
```

Después, en Antigravity: **File → Open Folder…** y elige la carpeta `prueba-rea`.

> Si en el futuro esa rama se fusiona en `main`, basta con quedarte en `main`.

COMPROBACIÓN: en el explorador de Antigravity ves `CLAUDE.md`, `.claude/` y `docs/`.

> **En pocas palabras**
> Trabaja siempre dentro de esta carpeta: así Claude cumple las reglas de `CLAUDE.md` (autorización, evidencias, sala limpia).

PASO 04

Instalar REA

La CLI y el servidor MCP

Es lo mismo que se hizo en la nube. `rea setup` te enseña los cambios antes de aplicarlos y hace copia de seguridad de lo que toca.

TERMINAL

```
# 1. CLI "rea" global
npm install --global rea-agents
rea --help

# 2. Registrar REA en Claude Code + instalar la skill
rea setup --client claude_code
#    (o, como en el curso:  npx rea-agents setup)
```

`rea setup` te preguntará qué agentes configurar; marca **Claude Code**. En macOS te ofrecerá además instalar **Hopper** (de pago); puedes decir que no y usar Ghidra.

COMPROBACIÓN

```
rea doctor
#  node:                       healthy
#  registration:claude_code:  healthy
#  ghidra:                     missing_analysis_engine   ← normal, lo arreglamos en el
paso 05
```

> **En pocas palabras**
>
> Un `npm install` y un `rea setup`. Con esto REA ya analiza apps JavaScript/Electron y .NET; para binarios nativos falta el motor.

PASO 05

Instalar Ghidra

El motor gratuito de desensamblado

REA busca Ghidra en la carpeta que indique la variable **`GHIDRA_INSTALL_DIR`**, y pide una release **12.1.x**. No hay instalador: se descarga un `.zip`, se descomprime y se apunta la variable a esa carpeta.

1. Abre `https://github.com/NationalSecurityAgency/ghidra/releases` y busca la release **Ghidra 12.1.x** (la más reciente de la serie 12.1).

2. En **Assets**, descarga `ghidra_12.1.x_PUBLIC_AAAAMMDD.zip` (≈ 500 MB).

3. Descomprímelo en una ruta **sin espacios ni acentos**, por ejemplo `C:\Tools\ghidra_12.1.x_PUBLIC` en Windows o `~/tools/ghidra_12.1.x_PUBLIC` en macOS/Linux.

4. Define `GHIDRA_INSTALL_DIR` apuntando a esa carpeta (la que contiene `ghidraRun`):

WINDOWS (PowerShell)

```
# Sustituye la ruta por la tuya exacta
setx GHIDRA_INSTALL_DIR "C:\Tools\ghidra_12.1.x_PUBLIC"
# Cierra Antigravity por completo y vuelve a abrirlo para que lo vea
```

MACOS (zsh)

```
echo 'export GHIDRA_INSTALL_DIR="$HOME/tools/ghidra_12.1.x_PUBLIC"' >> ~/.zshrc
source ~/.zshrc
# En macOS, si Gatekeeper bloquea los binarios descargados:
xattr -dr com.apple.quarantine "$GHIDRA_INSTALL_DIR"
```

LINUX (bash)

```
echo 'export GHIDRA_INSTALL_DIR="$HOME/tools/ghidra_12.1.x_PUBLIC"' >> ~/.bashrc
source ~/.bashrc
```

5. Opcional: abre Ghidra una vez a mano para comprobar que Java está bien (`ghidraRun.bat` en Windows, `./ghidraRun` en macOS/Linux). Si se abre la ventana, Java 21 funciona.

COMPROBACIÓN

```
rea doctor
#  ghidra:  healthy   (o ok: true)
```

> **En pocas palabras**
> Descargar el zip de Ghidra 12.1.x, descomprimirlo y apuntar `GHIDRA_INSTALL_DIR`. Reinicia Antigravity para que Claude Code herede la variable.

PASO 06

Verificar que Claude ve las herramientas

La prueba del /mcp

Reinicia Claude Code (sal con `/exit` o cierra la terminal) para que cargue el servidor MCP con la variable de Ghidra ya definida.

TERMINAL · CLAUDE CODE

```
claude
> /mcp
  rea: connected
    open_binary, binary_overview, analyze_function, decompile,
    list_procedures, search, xrefs, trace, inspect_artifact,
    analyze_javascript_application, inspect_managed_artifact ...
```

Si `rea` no aparece o sale como *failed*, ejecuta `rea setup --client claude_code` otra vez y reinicia Claude Code.

> **En pocas palabras**
> Si `/mcp` muestra `rea: connected` y `rea doctor` marca Ghidra en verde, la instalación está completa.

PASO 07

La primera prueba (sin riesgo)

Un programa tuyo, de principio a fin

Como recomienda la guía del curso, empieza con un binario **que hayas compilado tú**: así practicas los pasos 03 a 06 sin tocar software de terceros.

`pruebas/hola.c` (ya está incluido en el repositorio)

```
#include <stdio.h>
#include <string.h>

int checksum(const char *s) {
    int total = 0;
    for (size_t i = 0; i < strlen(s); i++) total = total * 31 + s[i];
    return total;
}

int main(int argc, char **argv) {
    const char *msg = argc > 1 ? argv[1] : "hola rea";
    printf("checksum(%s) = %d\n", msg, checksum(msg));
    return 0;
}
```

```
# macOS / Linux
gcc -O0 -o pruebas/hola pruebas/hola.c
# Windows: instala "MSYS2" o usa WSL, o compílalo con: zig cc -o pruebas\hola.exe
pruebas\hola.c
```

Prueba rápida desde la terminal (sin Claude):

```
rea decompile ./pruebas/hola main --provider ghidra --format json > main.json
rea function  ./pruebas/hola checksum --provider ghidra --format json
```

PROMPT · PRIMER CONTACTO (pégalo en Claude Code)

```
Quiero entender cómo funciona checksum en hola, que está en [RUTA ABSOLUTA A
pruebas/hola].
Es un programa mío, compilado por mí para practicar.

Primero, dime qué tipo de objetivo es (nativo, Electron/JS, .NET, Go, APK, WASM)
y elige la herramienta de REA adecuada para abrirlo. Todavía no empieces el
análisis profundo.

Después dame un mapa en un párrafo: con qué está construida la app, qué tamaño
tiene y dónde mirarías primero para encontrar checksum. Separa observaciones,
inferencias y desconocidos, y cita la evidencia de todo lo que afirmes.
```

PROMPT · ANOTACIÓN (después de decompilar)

```
Aquí tienes el pseudocódigo descompilado de checksum.

Para esta función:
1. Renombra cada variable con un nombre con significado según cómo se usa.
2. Explica su propósito en una frase.
3. Enumera las estructuras de datos que lee y escribe.
4. Nombra el patrón (singleton, factory, máquina de estados, observer...).
5. Reescríbela como C limpio e idiomático, con comentarios de documentación.
6. Señala cualquier falta de manejo de errores, validación de entradas o
   comprobación de seguridad.

Piénsalo con esfuerzo xhigh antes de responder: razona qué es cada valor antes
de decidir los nombres. Marca todo aquello de lo que no estés seguro.
```

Como tienes el código fuente original, puedes **comparar** lo que reconstruye Opus con `hola.c`: es la mejor forma de aprender a fiarte (o no) de cada resultado.

> **En pocas palabras**
>
> Compila algo tuyo, ábrelo con REA + Ghidra y repite los pasos 03–06 del curso. Cuando el flujo te salga solo, pasa a objetivos más grandes que estés autorizado a analizar.

Problemas típicos

Síntoma	Qué hacer
`npm` instala REA 6.7.0 en vez de la última	Tu Node es anterior a 22.23.4. Actualiza Node a la LTS actual y repite `npm install -g rea-agents`.
`rea: command not found` / "no se reconoce"	Cierra y abre la terminal. En Windows, comprueba que la carpeta global de npm (`npm prefix -g`) está en el `PATH`.
`rea doctor`: `GHIDRA_INSTALL_DIR is not set`	No reiniciaste Antigravity tras `setx`, o la variable apunta a la carpeta equivocada (debe contener `ghidraRun`).
Ghidra no arranca / error de Java	`java -version` debe decir 21. Si hay varias versiones, define `JAVA_HOME` apuntando al JDK 21.
`/mcp` no muestra `rea`	`rea setup --client claude_code` y reinicia Claude Code. Revisa que estás en la carpeta del repositorio.
`rea doctor` marca el host como no soportado en Windows	Usa **WSL2** (Ubuntu) y sigue la variante Linux dentro de WSL; Antigravity puede abrir carpetas de WSL.
macOS bloquea los ejecutables de Ghidra	`xattr -dr com.apple.quarantine "$GHIDRA_INSTALL_DIR"`.
Proxy corporativo / errores de certificados con npm	Configura `npm config set proxy …` y `npm config set cafile …` según tu red.

Mantener todo al día

```
npm update -g rea-agents      # REA saca correcciones a menudo
rea doctor                    # vuelve a comprobar tras cada actualización
claude update                 # Claude Code (si usaste el instalador nativo)
```

Para Ghidra, cuando salga una nueva 12.1.x: descarga el nuevo zip, descomprímelo al lado y cambia `GHIDRA_INSTALL_DIR` a la carpeta nueva.

Notas antes de lanzarte

1. Solo objetivos tuyos o autorizados. Es la primera regla del `CLAUDE.md`. En la UE (Directiva 2009/24/CE; en España, art. 100 de la Ley de Propiedad Intelectual) observar y probar el funcionamiento de un programa que usas legítimamente está permitido; **descompilar** solo en supuestos estrictos (básicamente interoperabilidad). Si vas a sacar un producto basado en esto, consulta a un abogado de propiedad intelectual.

2. El coste se dispara rápido. Esfuerzo `max`, enjambres de workers y binarios con miles de funciones consumen muchos tokens: resumen antes que volcado y prioriza por xrefs.

3. Hopper e IDA son opcionales. Hopper (de pago, ideal en Mac) e IDA (comercial) se integran igual con REA; Ghidra es gratuito y basta para todo el curso.

> **Por dónde empezar (sin riesgo)**
> Instala REA con Ghidra siguiendo esta guía, compila `pruebas/hola.c` y haz los pasos 03 a 06 del curso sobre él. Así aprendes el flujo completo sin tocar software de terceros.